

QT_Project 보고서

예비단원 최윤서

1. 프로젝트 개요

a. 프로젝트 선정 배경

- 최근 기숙사에서 세미나실 자리 독점을 검사한다는 문자를 받음
- 과연 자리 독점이 제대로 잡힐 지 의문이 들었음
- 1층 업무 관리도 해야하는 경비아저씨가 모든 업무를 맡기엔 무리가 있을거라고 생각
- 자율로 돌아다니는 로봇이 있으면 좋을거같다는 생각에서 착안

b. 프로젝트 목적

- 무인으로도 자습실 좌석의 사용 상태 지속적 관리
- 소지품이 남아있는 좌석을 자동으로 탐지 → 순찰로봇이 효율적으로 확인
- 궁극적으로 자습실의 사석화를 막음

c. 시스템 개요

- `occupied = true`: 사용자가 현재 좌석에 있음
- `occupied = false`: 사용자가 현재 좌석에 없음
- `has_belongs = true`: 좌석에 개인 물품이 있음
- `has_belongs = false`: 좌석에 개인 물품이 없음

2. 핵심 기능 및 알고리즘

a. 좌석 상태 관리

- 각 좌석은 `Seat` 클래스를 이용하여 독립적으로 관리
- `Seat` 객체 → 좌석 번호, 좌석 위치, 사용 여부, 소지품 여부, 이석 시간, 패널티 처리 여부 저장

```

class Seat{
private:
    int seat_id; //좌석 번호
    bool occupied; //사람이 있는지 없는지 판단
    int absence_start_time; //좌석에서 이탈한 시간
    bool has_belongings; //짐이 남아있는지 없는지 체크

    double x,y;
    bool penalty_processed; //벌점이 들어갔는지 안들어갔는지 체크

public:
    Seat(int id, double pose_x, double pos_y); //생성자

    int getSeatID(); //ID 받아오기
    bool isOccupied(); //사람 있는지 없는지 판단
    bool hasBelongings();
    int getAbsenceStartTime(); //좌석에서 이탈한 시간

    void getPosition(double& pos_x, double& pos_y); //좌석 위치 조회

    void updateStatus(bool new_occupied, bool new_has_belongings); //좌석 상태 변경
    bool isPenaltyTarget(); //벌점 대상 여부 확인

    void processPenalty();
    bool isPenaltyProcessed();
};

#endif

```

b. 이석 판단 알고리즘

- 이석의 판단 조건: 사람 X and 짐 O and 이석 시간 20초 이상

```

// 사람이 있는 상태에서 사람이 나갔고,
// 짐이 남아 있는 경우
if (occupied == true && new_occupied == false && new_has_belongings == true)
{
    // 짐을 두고 이석한 순간부터 시간 측정
    absence_start_time = std::time(nullptr);
}

```

- 만약 10초 동안 자리를 비웠다면 순찰 대상 X
- 20초 이상 상대 지속되고 위 조건을 만족하면 순찰대상 O

c. 순찰 대상 선정 방식

- PatrolPlanner 클래스에서 담당
- 순찰 대상 조건: 1. 사용자가 좌석에 없어야한다
2. 좌석에 소지품이 있어야한다
3. 이석 시간이 20초 이상이어야한다
4. 현재 이석 상황에 대해 패널티 처리가 완료 X
5. 현재 순찰에서 제외된 좌석이 아니어야함

d. TSP 기반 순찰 경로 알고리즘

- 순찰 대상이 1,3,5 번이라고 하더라도 단순히 1 -> 3 -> 5 순서로 이동하는 것은 아님
- 여러 좌석을 한 번씩 방문 → 전체 이동 거리를 줄이는 문제를 TSP 형태로 모델링
- TSP: 여러 지점을 한번 씩 방문하면서 전체 이동 비용을 최소화하는 경로를 찾는 문제
- Nearest Neighbor
 - 가장 가까운 미방문 좌석을 다음 목적지로 선택하는 방식
 - 1) 출발 위치 설정
 - 2) 현재 위치에서 가장 가까운 미방문 좌석 탐색
 - 3) 해당 좌석 방문
 - 4) 현재 위치 변경
 - 5) 다시 가장 가까운 미방문 좌석 탐색
 - 6) 모든 좌석 방문
- 2-opt
 - 두개의 간선을 선택 → 경로의 일부를 뒤집어 전체 이동거리가 감소하는지 확인하는 방식
- 최종 알고리즘
 - 1) 순찰 대상 좌석 확인
 - 2) 좌석 위치 좌표 확인
 - 3) Nearest Neighbor로 초기 경로 설정
 - 4) 2-opt 로 경로 일부 변경
 - 5) 변경된 경로의 이동 거리 계산
 - 6) 기존 경로보다 짧으면 변경 적용
 - 7) 더 이상 개선되지않을 때까지 반복
 - 8) 최종 순찰 경로 생성

e. 순찰 및 패널티 처리과정

- 10초마다 한번 씩 순찰 수행
 - 10초가 되면 패널티 조건을 만족하는 좌석들을 확인하고 해당 좌석 순찰 대상으로 지정
- 1) 10초 순찰 주기 도달
 - 2) 현재 순찰 대상 확인
 - 3) 순찰 대상 좌석 목록 생성
 - 4) GetRoute Service 요청
 - 5) Nearest Neighbor
 - 6) 2-opt
 - 7) 최종 순찰 경로 반환
 - 8) 로봇 이동 시작
 - 9) 첫 번째 좌석 방문
 - 10) 현재 좌석 상태 재확인
 - 11) 패널티 조건 만족하면 패널티 처리 → 다음 좌석 방문
 - 12) 패널티 조건 만족 안하면 다음 좌석 방문
 - 13) 원위치 복귀
 - 14) 다시 10초 대기

3. ROS2 및 QT 구현

a. Topic 통신

- 본 프로젝트에서는 총 3개의 topic을 사용
- 1) /study_room/seat_status : 좌석 상태 전달
 - 2) /study_room/penalty_seats : 현재 패널티 대상 좌석 전달
 - 3) /stud_room/seat_command : GUI에서 좌석 상태 변경 명령 전달

b. Service 통신

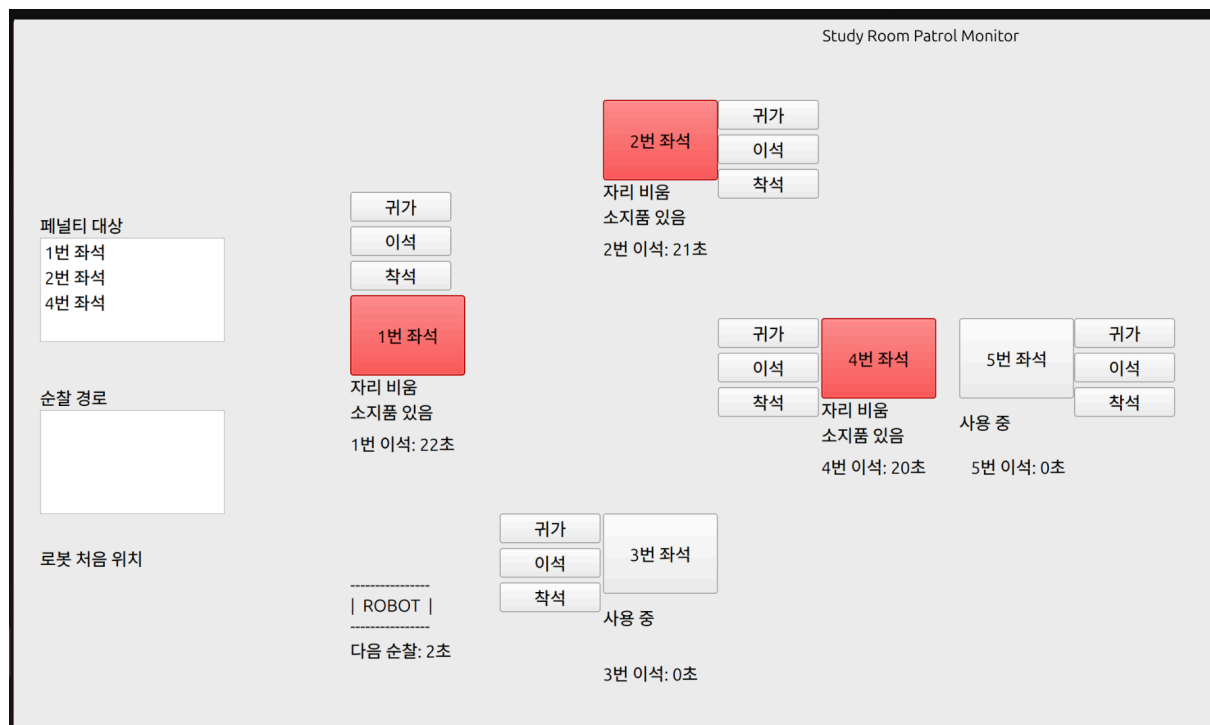
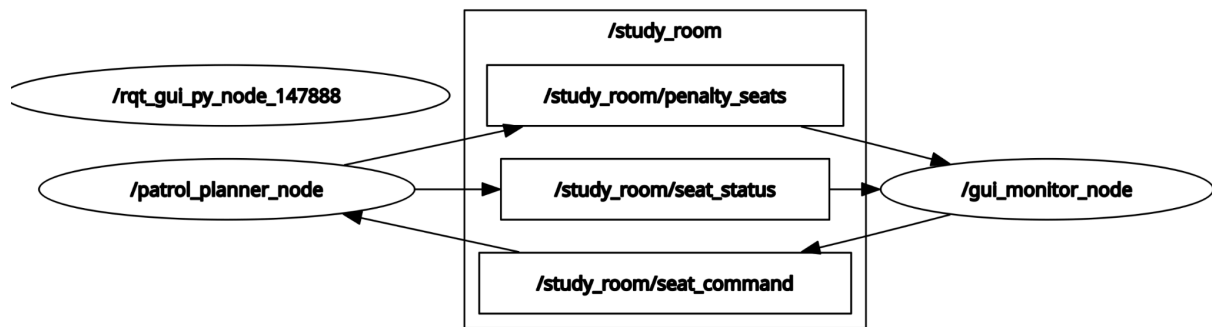
- 본 프로젝트에서는 총 2개의 Service를 사용
 - topic은 지속적인 상태 전달에 사용, service는 특정 요청에 대한 응답이 필요할 때 사용
- 1) /patrol/get_route : 순찰 대상 좌석 목록 전달 → TSP 기반 알고리즘 → 순찰 경로 반환
 - 2) /patrol/process_penalty: 로봇이 실제 좌석 확인 → 패널티 처리 필요한 좌석 반환

c. 예외 처리 및 상태 초기화

- 순찰 대상이 없는 경우에 불필요하게 경로 계산 X → 다음 순찰 주기 대기
- Service 요청으로 전달된 좌석 ID가 실제 존재하는 좌석인지 확인 → 유효하지않다면 ID가 전달되어도 정상적인 순찰 경로에 포함하지않음
- 상태 초기화" 사용자가 다시 좌석에 앉으면 해당 좌석의 이석 시간 초기화

4. 구현 결과 및 결론

a. 프로그램 작동 결과



Study Room Patrol Monitor

페널티 대상

순찰 경로

1번 좌석
2번 좌석
4번 좌석

로봇 처음 위치

귀가
이석
착석

1번 좌석

자리 비움
소지품 있음
1번 이석: 9초

귀가
이석
착석

2번 좌석

자리 비움
소지품 있음
2번 이석: 46초

귀가
이석
착석

3번 좌석

자리 비움
소지품 있음
3번 이석: 6초

귀가
이석
착석

4번 좌석

사용 중
4번 이석: 0초

귀가
이석
착석

5번 좌석

자리 비움
소지품 있음
5번 이석: 4초

| ROBOT |

다음 순찰: 5초

Study Room Patrol Monitor

페널티 대상

3번 좌석
5번 좌석

순찰 경로

1번 좌석
3번 좌석

로봇 위치: 3번 좌석

귀가
이석
착석

1번 좌석

자리 비움
소지품 있음
1번 이석: 27초

귀가
이석
착석

2번 좌석

자리 비움
소지품 있음
2번 이석: 64초

귀가
이석
착석

3번 좌석

자리 비움
소지품 있음
3번 이석: 24초

귀가
이석
착석

4번 좌석

사용 중
4번 이석: 0초

귀가
이석
착석

5번 좌석

자리 비움
소지품 있음
5번 이석: 22초

| ROBOT |

순찰 중